

RESEARCH SEMINAR

SimToolReal

An Object-Centric Policy for
Zero-Shot Dexterous Tool Manipulation

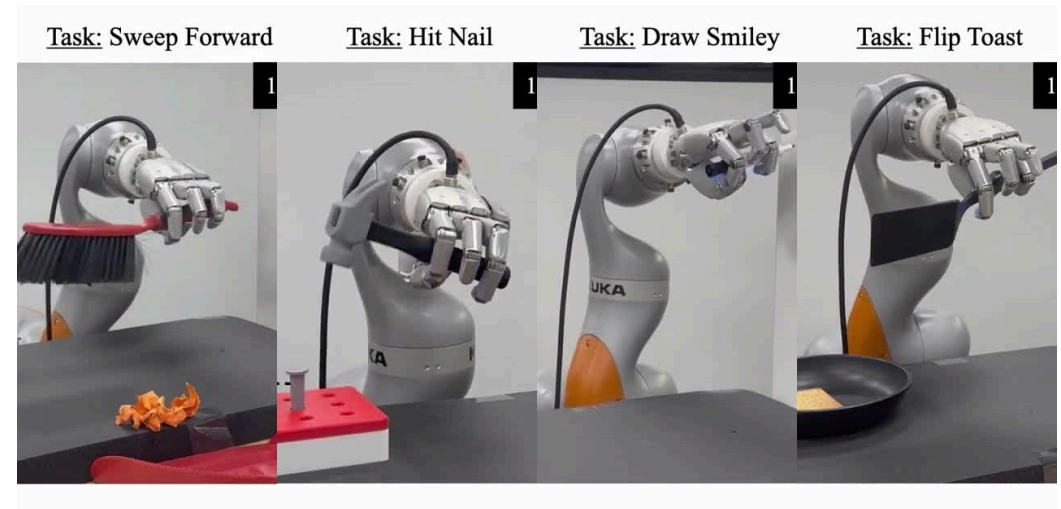
Can one reusable controller
turn tool motion into robot skill?

Kushal Kedia* · Tyler Ga Wei Lum*

Jeannette Bohg · C. Karen Liu

Cornell University · Stanford University

RSS 2026 · Based on arXiv v2, 24 February 2026



Real-world experiments: brush, hammer, marker and spatula

How would you teach a robot to use tools?

Suppose the robot must use
a new brush, hammer or spatula.
What would you build first?

Collect robot demonstrations

Learn a policy from action supervision.

Use a human demonstration

Transfer the motion across embodiments.

Learn through simulation

Let physical interaction shape the controller.

What makes demonstration-based tool use difficult?

Thin-object grasping, in-hand rotation and external forces make contact quality important.

Robot action demonstrations

- High-quality dexterous teleoperation is difficult.
- Limited force feedback makes stable contacts hard to demonstrate.

Human motion retargeting

- Human and robot fingers have different kinematics.
- Matching hand motion does not ensure a physically stable grasp.

What if a demonstration specified tool motion, while the robot learned how to realize it?

Tool use as a sequence of pose-reaching problems

TASK SPECIFICATION

Human tool motion

6D pose waypoints



Reusable goal-reaching
RL policy



ROBOT EXECUTION

Arm + hand actions

Specify where the tool goes

3D position + 3D orientation

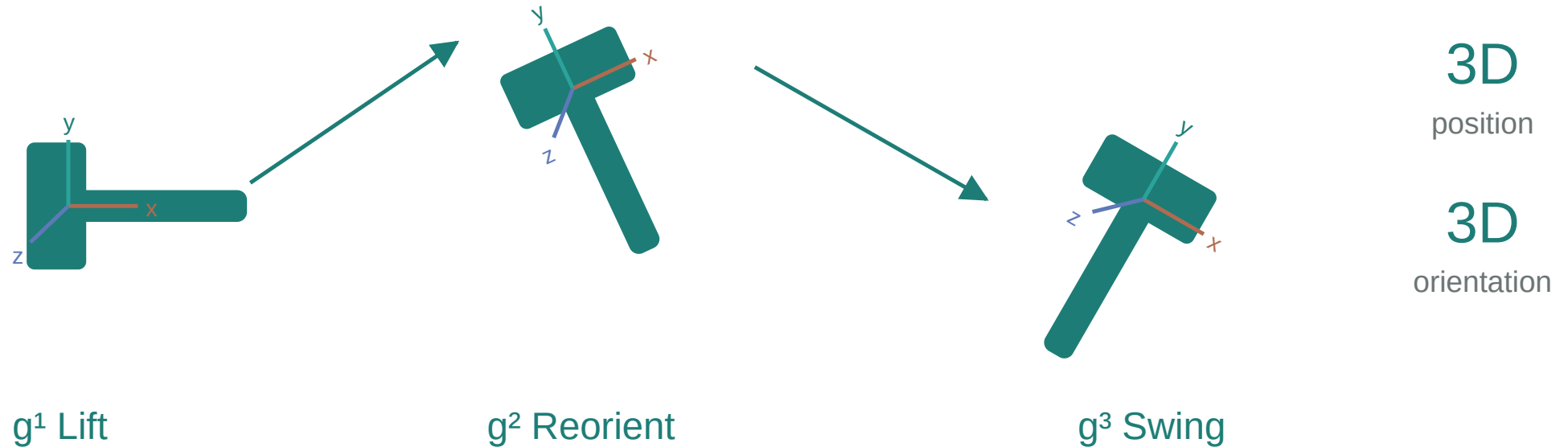
Learn how this robot moves it

Contacts and regrasping emerge from RL

Object goals specify the motion. RL discovers robot-specific grasps and in-hand actions.

A 6D pose specifies position and orientation

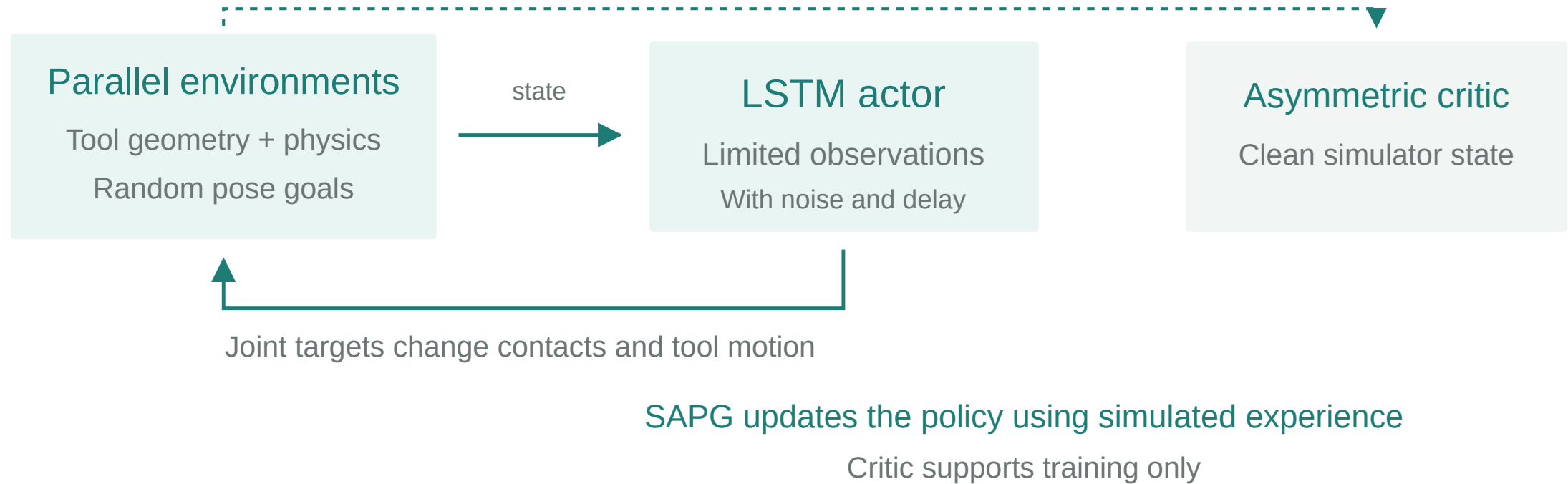
A task becomes an ordered set of rigid-object configurations



$$g^k = (R^k, p^k) \in SE(3), \quad \mathcal{G} = \{g^1, \dots, g^K\}$$

Training pipeline: learn pose reaching through interaction

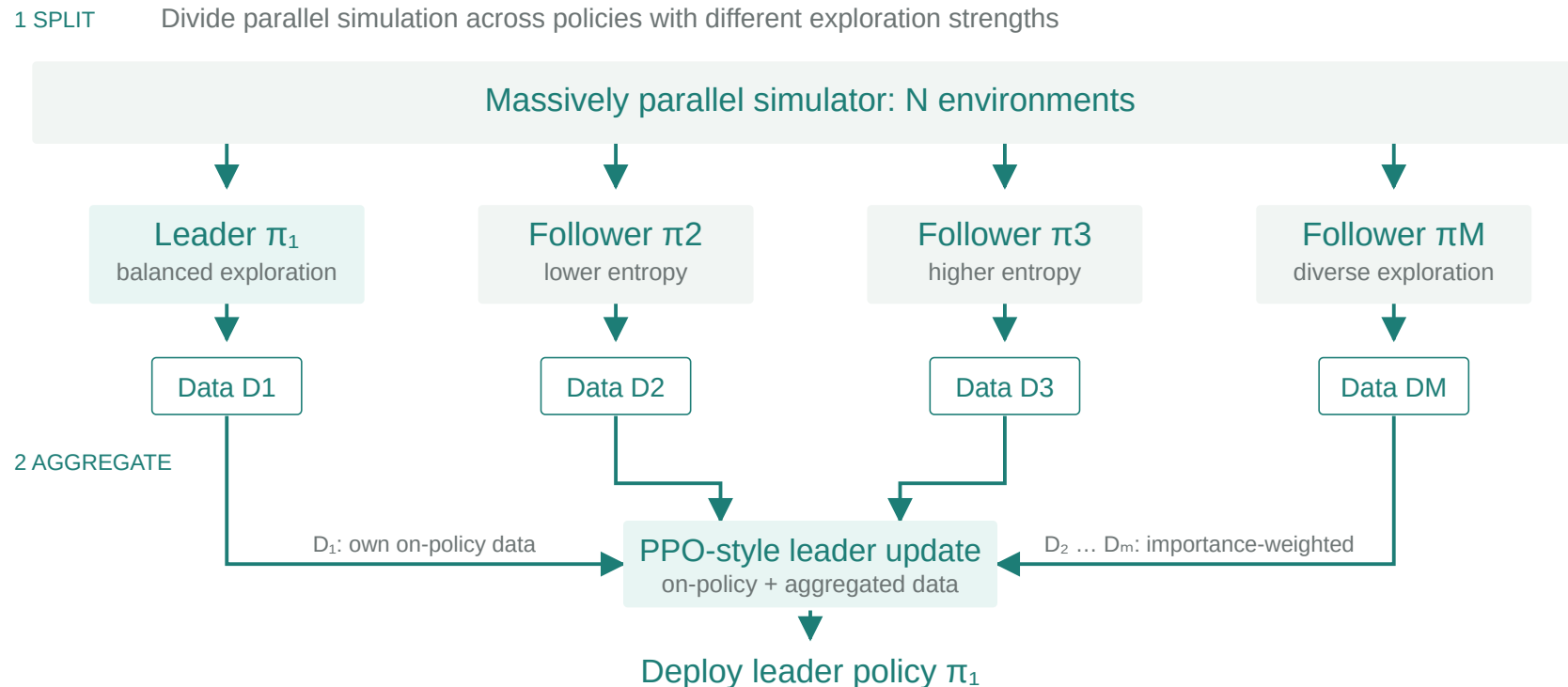
SIMULATION ONLY



Procedural tools and random goals supply the training distribution. No human task trajectories enter this loop.

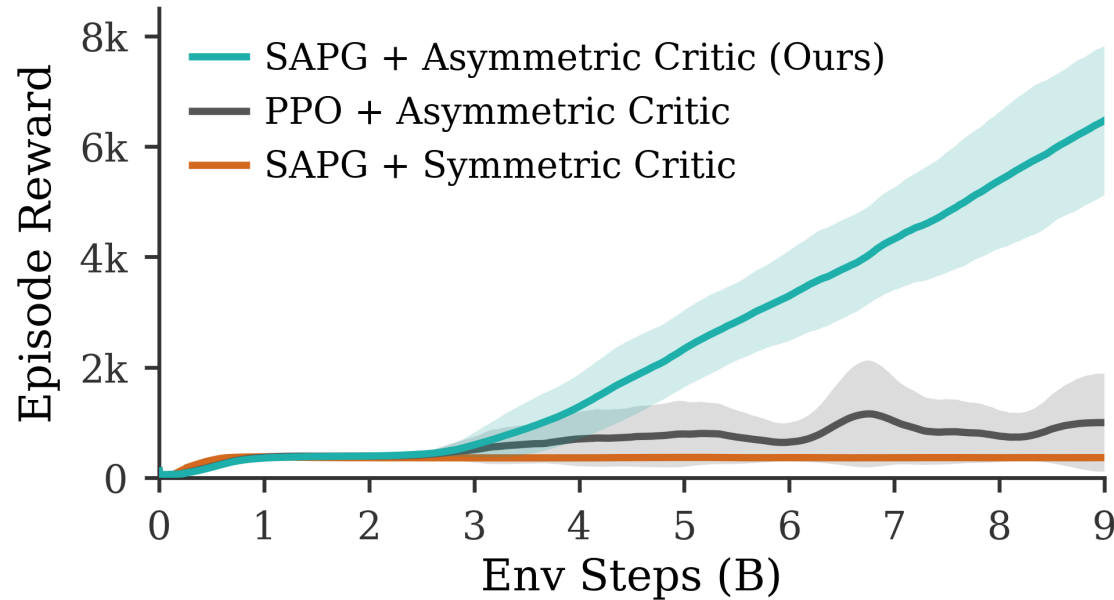
SAPG splits exploration and aggregates experience

One PPO policy can collect similar experience across thousands of parallel environments.



SAPG retains PPO-style actor–critic updates while changing how experience is explored and reused.

SAPG learns more effectively than PPO in this setting



Paper Fig. 8 · Training reward, mean over five seeds

Controlled comparison

- Both runs retain the asymmetric critic.
- Replacing SAPG with PPO changes the exploration and data aggregation scheme.

Observed result

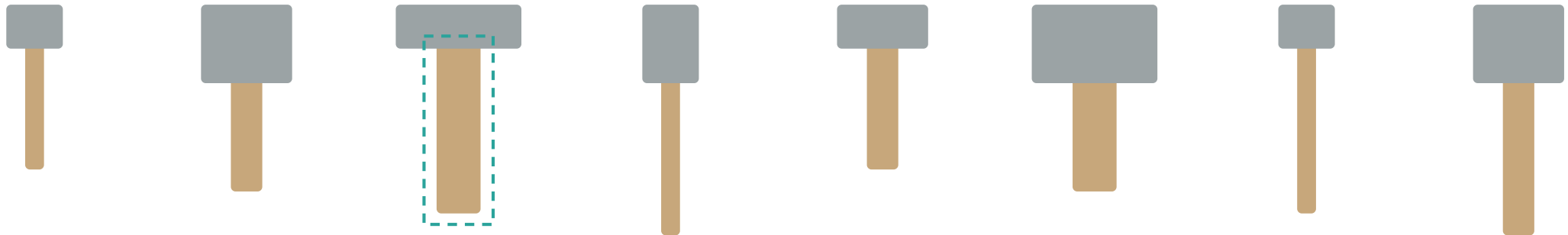
- SAPG continues improving across 9B environment steps.
- PPO reaches a much lower plateau under the matched setup.

The symmetric-critic ablation also performs poorly, so the result supports the combination rather than SAPG alone.

Metric: simulation training reward, not real-world functional task completion.

Procedural tools cover shape and mass variation

One structural prior, many physical instances



Geometry

Shape + size

Mass distribution

Handle / head densities

Shared policy inputs

Pose + grasp-region box

The commonality is a geometric interface: tool pose and a graspable region.

RL training signal: how each reward term is constructed

$$r = r_{\text{smooth}} + r_{\text{grasp}} + \mathbb{I}_{\text{grasped}} r_{\text{goal}}$$

1 · Smooth motion

$$r_{\text{smooth}} = -\lambda_{\text{arm}} \|\dot{q}_{\text{arm}}\|_1 - \lambda_{\text{hand}} \|\dot{q}_{\text{hand}}\|_1$$

L1 penalties on current arm and hand joint velocities.

2 · Acquire the grasp

$$r_{\text{grasp}} = r_{\text{approach}} + (1 - \mathbb{I}_{\text{grasped}}) r_{\text{lift}}$$

$$r_{\text{approach}} = \lambda_{\text{approach}} \max(d_{\text{ft}}^* - \bar{d}_{\text{ft}}, 0)$$

Improvement in mean fingertip-to-object distance.

$$r_{\text{lift}} = \lambda_{\text{lift}} \max(z - z_{\text{init}}, 0) + B_{\text{lifted}} \mathbb{I}[z \geq z_{\text{lifted}}]$$

Height progress plus a one-time lifted bonus.

3 · Reach each goal pose

$$r_{\text{goal}} = \max(d^* - d(o_t, g), 0) + B_{\text{succ}} \mathbb{I}[d(o_t, g) < \epsilon]$$

Progress: reward improvement over the best distance so far.

Success: bonus below ϵ , then sample the next goal.

Pose distance: $d(o_t, g) = \max_{i=1, \dots, 4} \|o_{t,i} - g_i\|_2$. Four fixed-scale keypoints jointly encode translation and rotation.

These are environment reward terms. SAPG still optimizes an actor–critic objective from the resulting returns.

LSTM memory carries interaction history across steps

THE SAME RECURRENT NETWORK AT EVERY CONTROL STEP



Cell state c_t : internal memory

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Forget gate f_t retains old information. Input gate i_t writes candidate memory.

Hidden state h_t : current representation

$$h_t = o_t \odot \tanh(c_t)$$

Output gate o_t exposes information to the action MLP and the next time step.

Both states have 1,024 dimensions. They carry a learned summary of history rather than a stack of raw past frames.

The action head controls the arm and hand differently

$$a_t = [a_t^{\text{arm}}, a_t^{\text{hand}}] \in [-1, 1]^{29}$$

7 arm joints: incremental targets

$$\hat{q}_t^{\text{arm}} = q_{t-1}^{\text{target, arm}} + 0.025a_t^{\text{arm}}$$

Each action adjusts the previous target.

22 hand joints: absolute targets

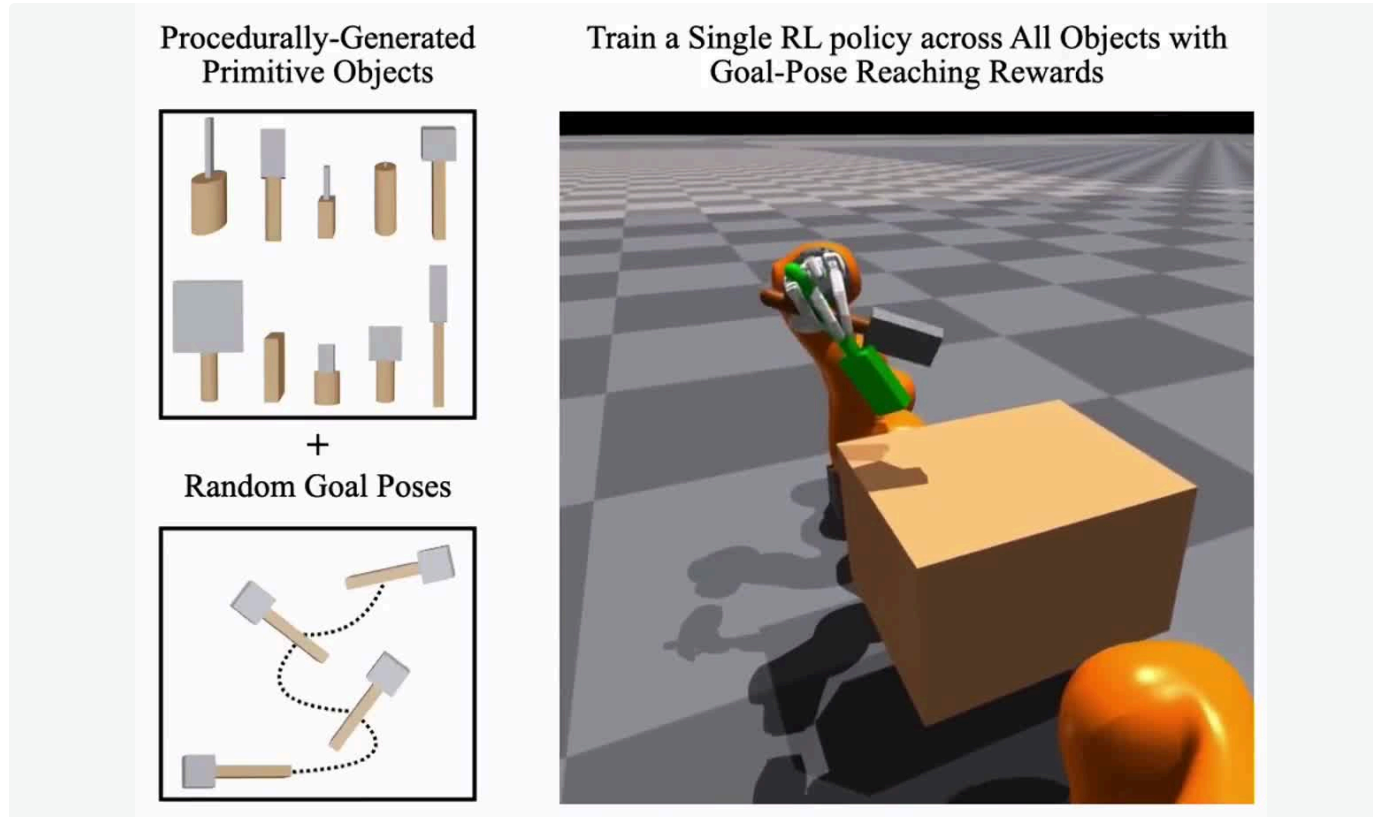
$$\hat{q}_t^{\text{hand}} = q_{\min} + \frac{a_t^{\text{hand}} + 1}{2} \odot (q_{\max} - q_{\min})$$

Each action maps into the joint's allowed range.

Joint limits and EMA smoothing ($\alpha = 0.1$) produce the final targets at 60 Hz.

The deployed actor keeps its weights fixed. Recurrent state updates at every control step.

Simulation: reaching diverse tool poses



WHAT TO WATCH

The **green mesh** indicates the current target pose.

The policy learns to acquire and reorient procedurally generated tools.

There is no human task trajectory in this training loop.

SimToolReal project: training environment · looping GIF · silent · 1× speed

[Original project video ↗](#)

Simulation exposes the state. Reality requires estimation.

Inside the simulator

- Exact tool pose and geometry are available.
- Random goal poses define the task.
- Noise and delays train the actor on imperfect inputs.

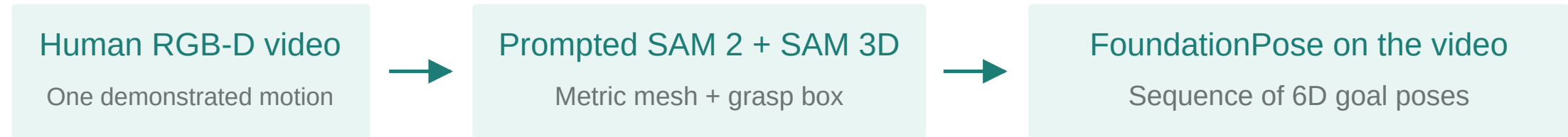
On the real robot

- The tool pose is hidden behind pixels and occlusion.
- A human demonstration supplies a meaningful trajectory.
- Perception must recover the same policy interface.

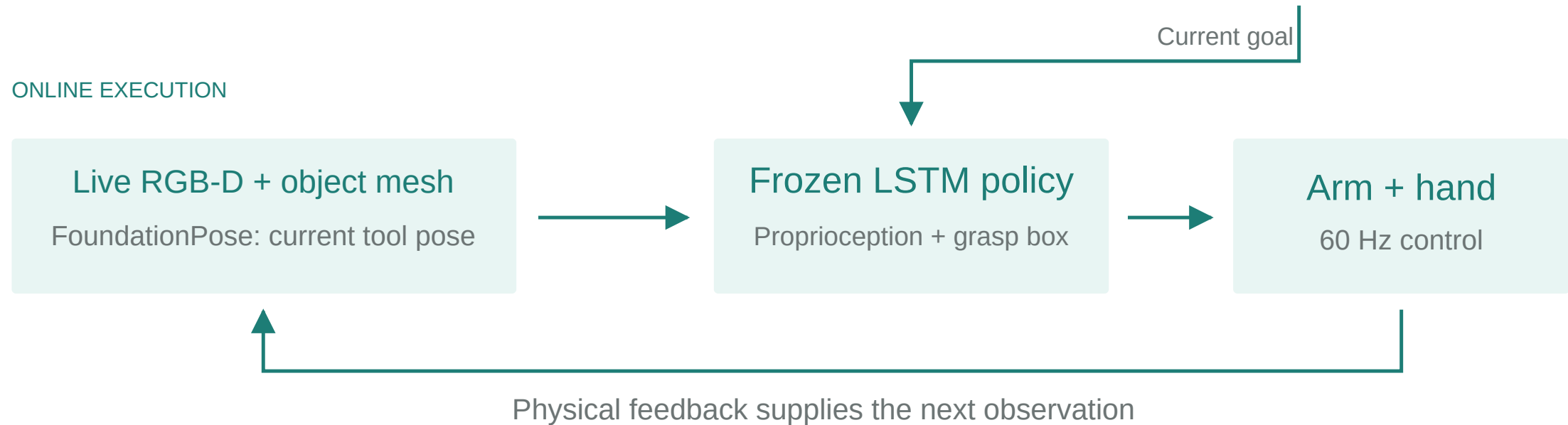
Keep the motor policy fixed. Replace simulator-derived inputs with measured and estimated inputs.

Real deployment supplies the same policy interface

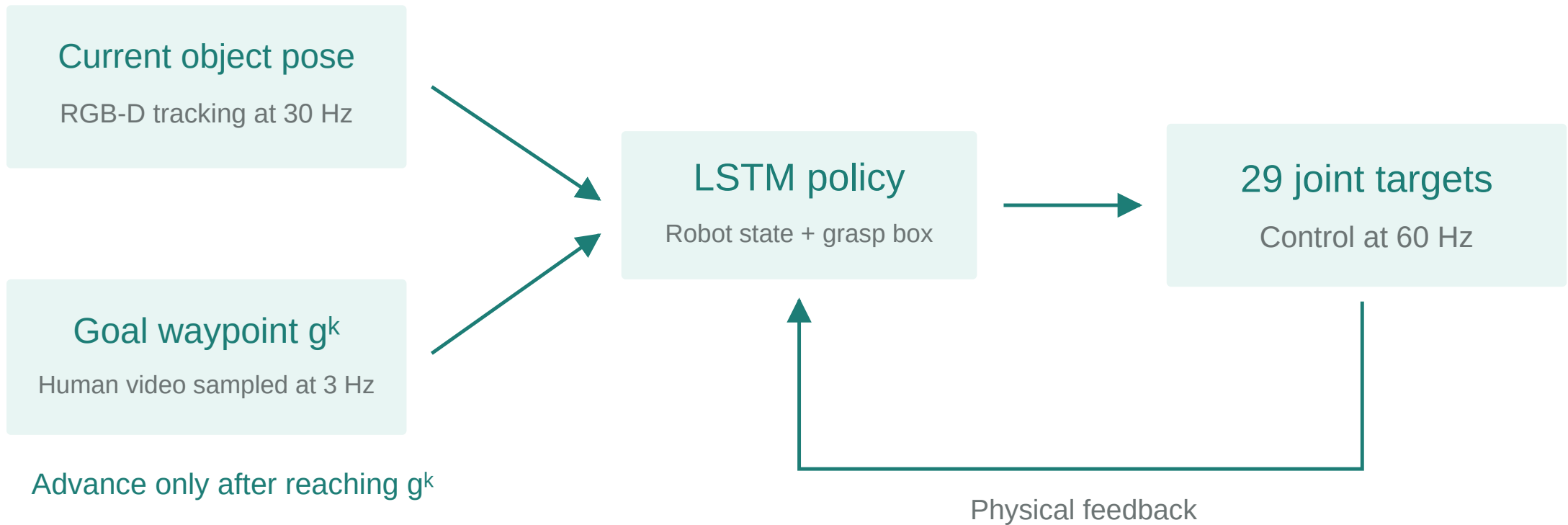
SETUP FOR A NEW TOOL / DEMONSTRATION



ONLINE EXECUTION

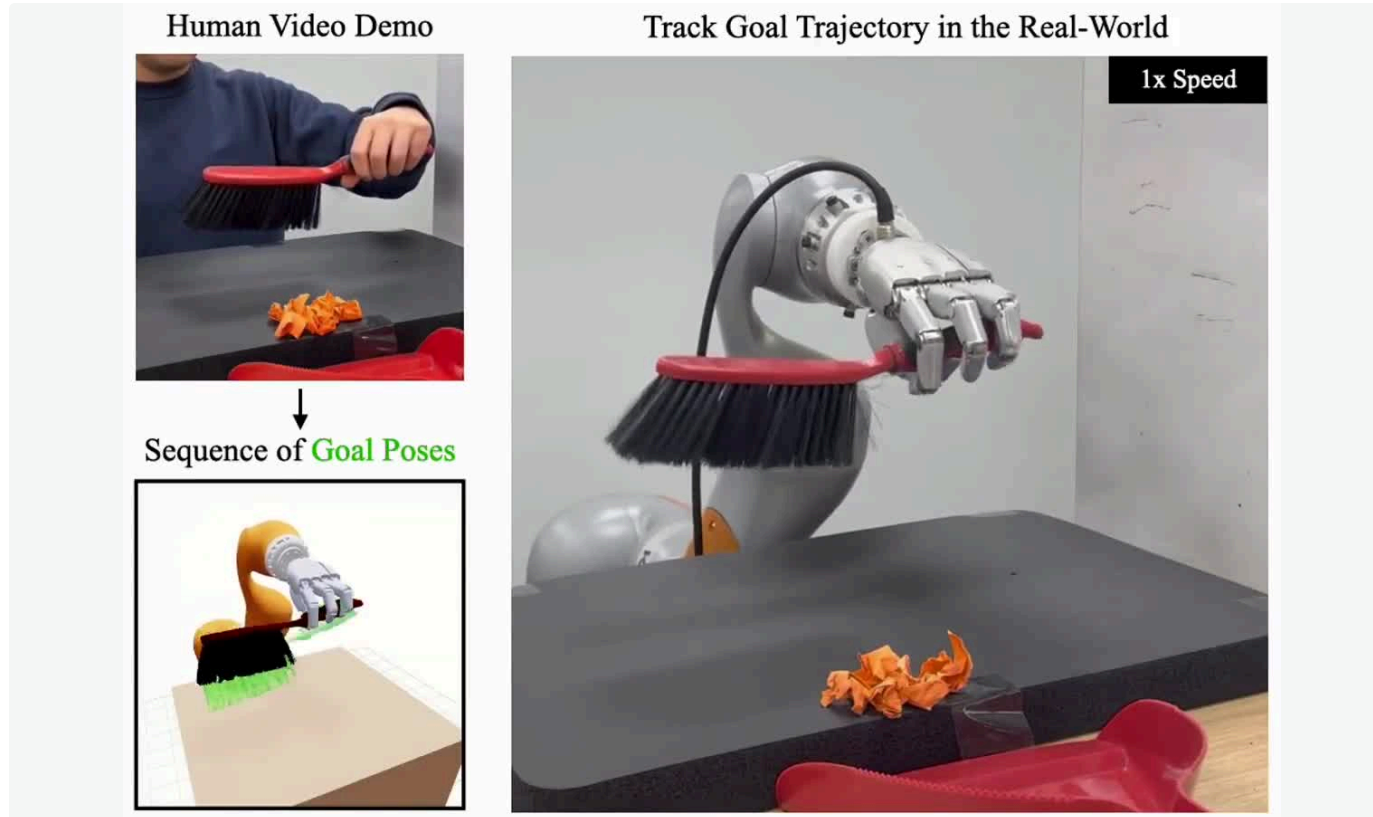


The robot advances goals only after reaching them



3 Hz is waypoint sampling. Goal switching is event-driven, while the controller runs at 60 Hz.

The closed loop follows unseen tool trajectories



SimToolReal project: real-world inference · looping GIF · silent · 1x speed

[Original project video ↗](#)

WHAT TO WATCH

The human demonstration supplies the **tool-motion goals**.

The fixed policy grasps, reorients and moves the real brush.

The goal interface specifies tool motion rather than robot finger trajectories.

Brush and hammer: reorientation before interaction

Brush

Brush (Zero-Shot)

Task: Sweep Forward



Task: Sweep Right



Brush · looping GIF · silent · 1× speed

[Original project video ↗](#)

Reorient the handle, then sweep along the surface.

Hammer

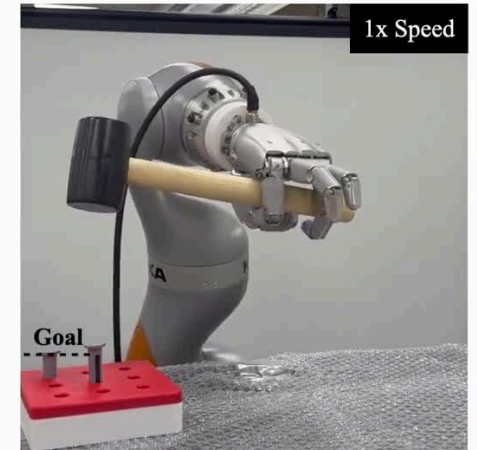
Hammer (Zero-Shot)

Task: Hit Nail



Hammer · looping GIF · silent · 1× speed

Task: Hit Nail



[Original project video ↗](#)

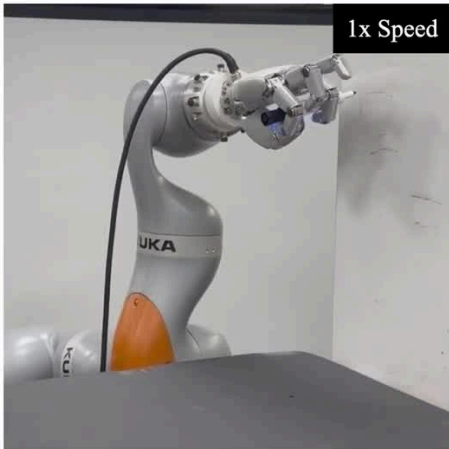
Change the grasp-tool orientation before the swing.

Marker and eraser: tool motion along a surface

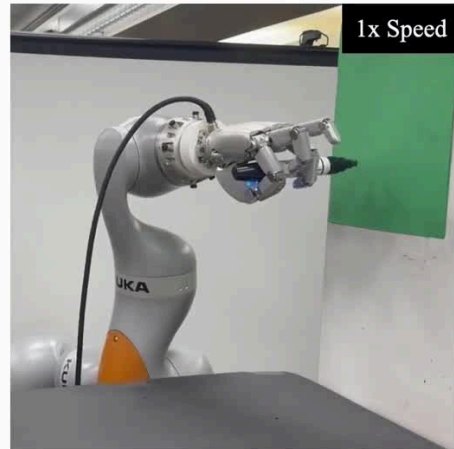
Marker

Marker (Zero-Shot)

Task: Draw Smiley



Task: Draw Smiley



Marker · looping GIF · silent · 1× speed

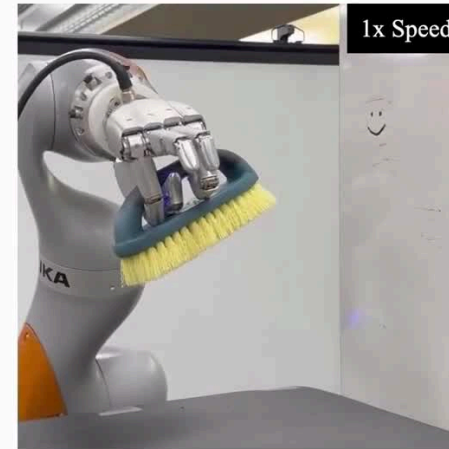
[Original project video ↗](#)

Acquire a thin tool and move its tip across the board.

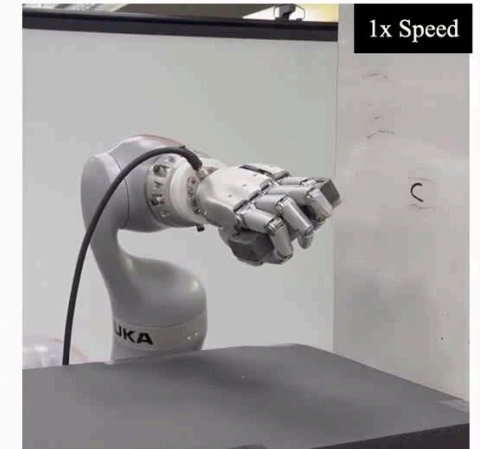
Eraser

Eraser (Zero-Shot)

Task: Wipe Whiteboard



Task: Wipe Whiteboard



Eraser · looping GIF · silent · 1× speed

[Original project video ↗](#)

Maintain a grasp while following a wiping path.

Spatula and screwdriver: larger orientation changes

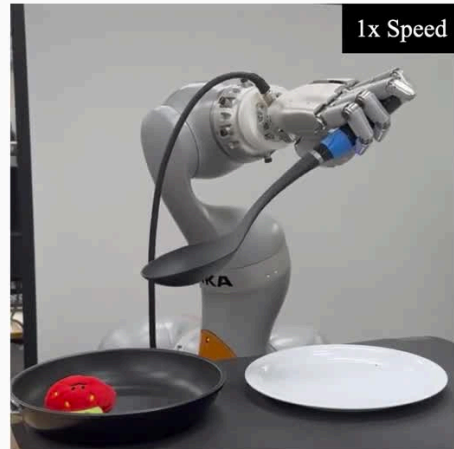
Spatula

Spatula (Zero-Shot)

Task: Flip Toast



Task: Serve Strawberry



Spatula · looping GIF · silent · 1× speed

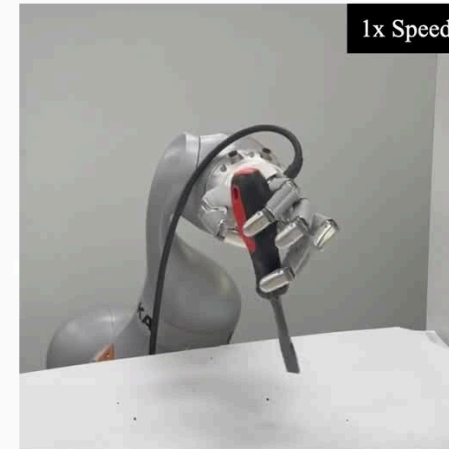
[Original project video ↗](#)

Serving and flipping demand changing tool orientation.

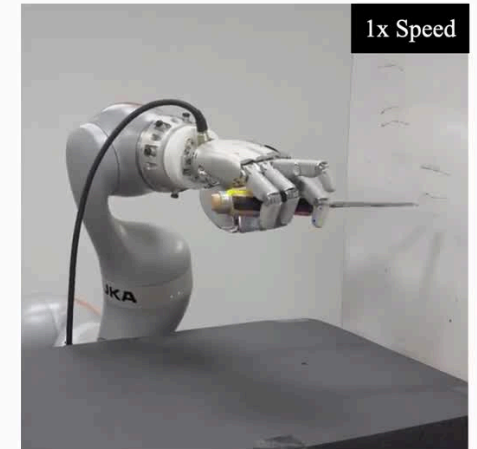
Screwdriver

Screwdriver (Zero-Shot)

Task: Spin Vertically



Task: Spin Horizontally



Screwdriver · looping GIF · silent · 1× speed

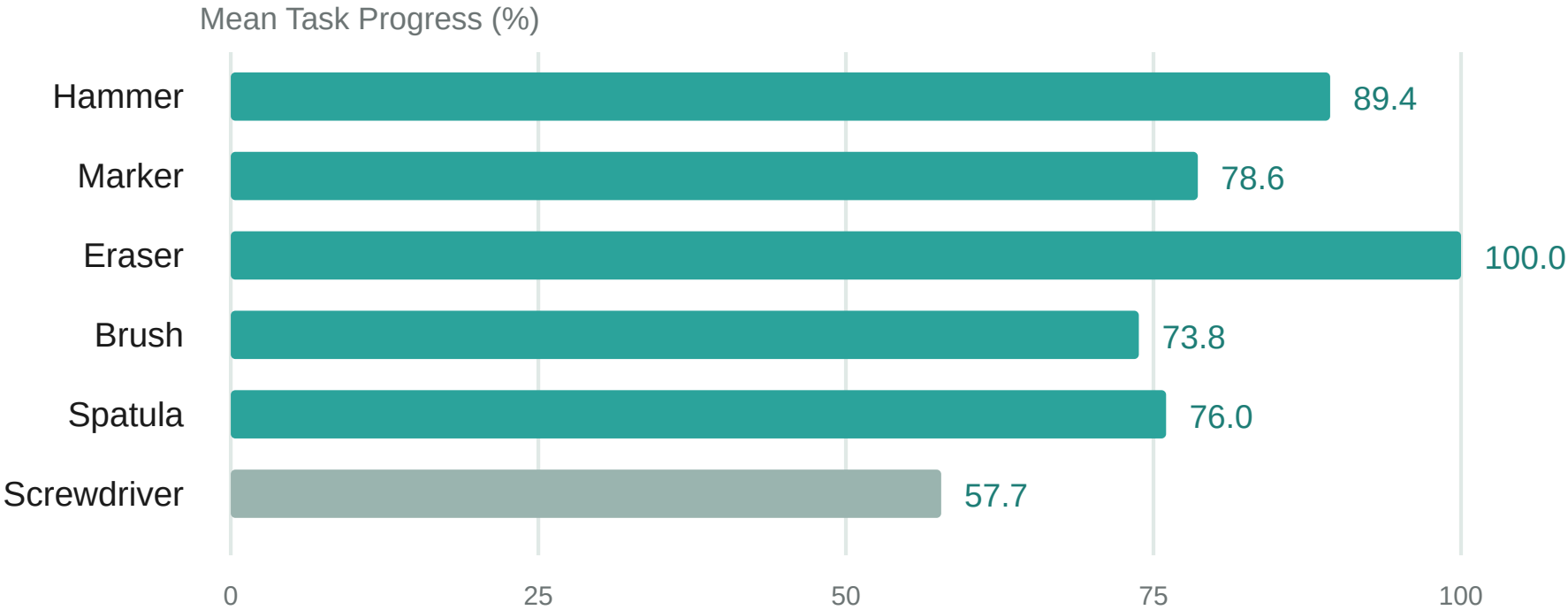
[Original project video ↗](#)

Reorient and spin the tool in free space.

Real-world transfer across six tool categories

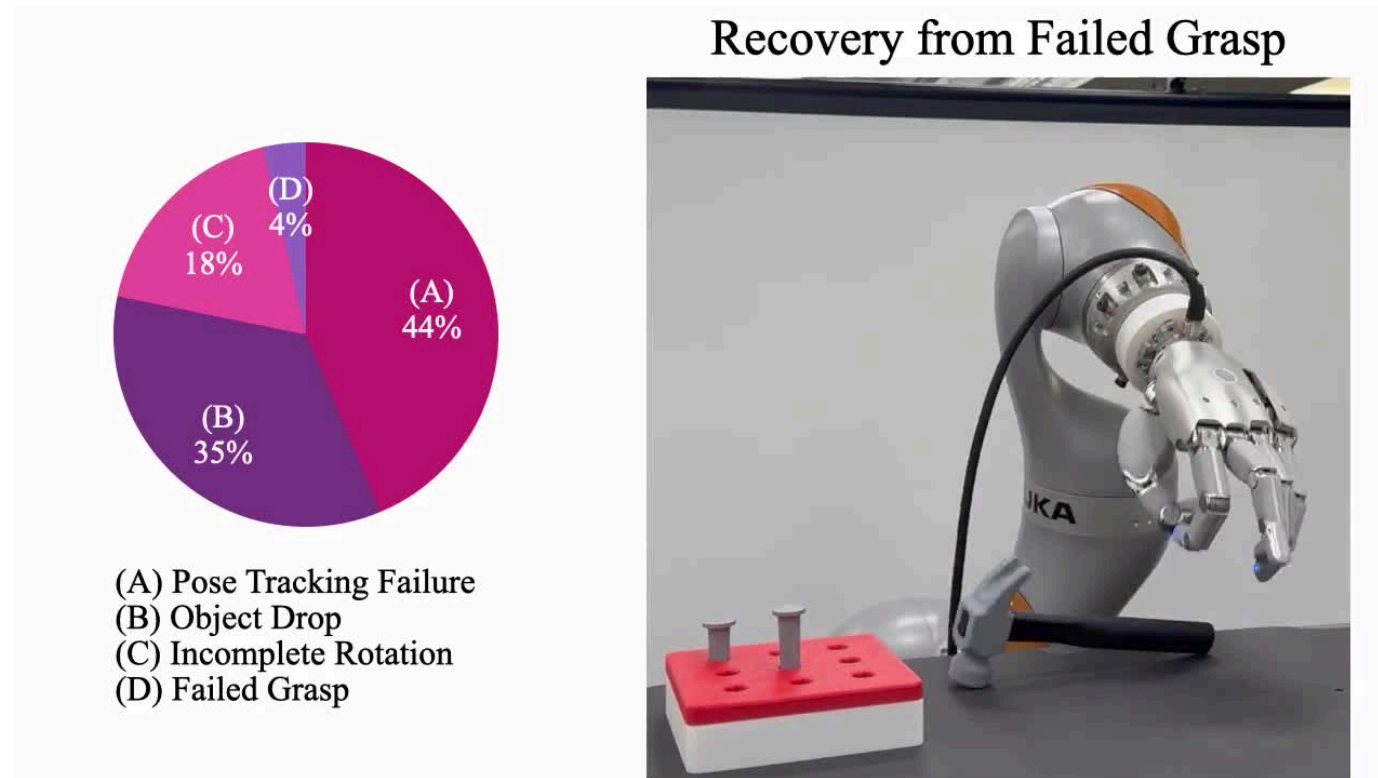
12 objects · 24 object–task variations · 120 rollouts

Task Progress = fraction of sequential pose goals reached.



Mean progress: **79.3%**. This metric does not establish functional task success.

Recovery illustrates feedback, with clear limits



SimToolReal project: recovery behavior · looping GIF · silent · 1× speed

[Original project video ↗](#)

WHAT TO WATCH

The policy can attempt to **regrasp** after an error.

Recovery depends on the tool staying **reachable and trackable**.

Rigid pose goals leave force objectives, clutter and task replanning unresolved.

Structured simulation as a route to reusable robot skills

PRESENTER REFLECTION · NOT RESULTS ESTABLISHED BY THIS PAPER

01 · PRIMARY TAKEAWAY

Beyond scaling end-to-end action models, **structured simulation** may offer another path toward skill acquisition in the wild.

RL learns reusable motor competence. Perception, demonstrations or a higher-level model specify what the tool should do.

02 Compact interfaces beyond tool use

Other interactions may admit a useful task interface, although pose alone will not always suffice. Contact, force, articulation or shape may need to enter the state.

03 Lightweight VLMs for semantic setup

A VLM could select tools, grasp regions and subgoals, while geometric perception retains metric, low-latency tracking.

SimToolReal demonstrates the motor-skill component inside a designed interface. Autonomous task and goal discovery remain open.

References, videos and source material

Main paper

Kedia, Lum, Bohg and Liu. *SimToolReal: An Object-Centric Policy for Zero-Shot Dexterous Tool Manipulation*.
arxiv.org/abs/2602.16863v2 · [RSS 2026 proceedings](#)

Project and code

simtoolreal.github.io · github.com/tylerlum/simtoolreal

Closely related work

Chen et al. *Object-Centric Dexterous Manipulation from Human Motion Data*. CoRL 2024.

Lum et al. *Crossing the Human-Robot Embodiment Gap with Sim-to-Real RL Using One Human Demonstration*. 2025.

Singla et al. *Split and Aggregate Policy Gradients*. ICML 2024.

Petrenko et al. *DexPBT*. 2023.

Design reference

[Mizoreww/steerable-vla-talk](https://mizoreww.github.io/steerable-vla-talk) and the previous A2A seminar deck.